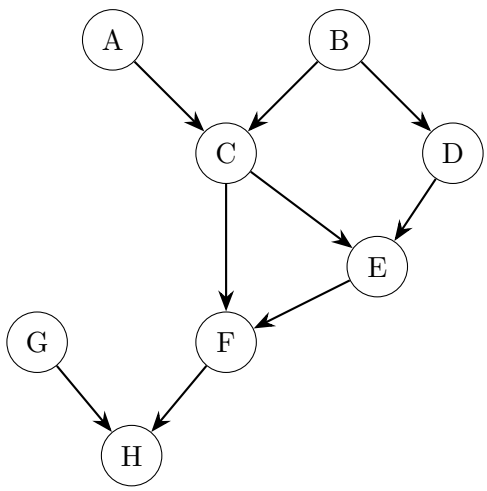
HWK#1 Part A

Shreesh Karjagi

Problem 1

From Bayesian Network in Figure 1.



Part (i)

by using the chain rule for Bayesian networks each node is conditioned on its parents:

$$\begin{aligned} P(A,B,C,D,E,F,G,H) = P(A) \cdot P(B) \cdot P(G) \cdot P(C \mid A,B) \cdot P(D \mid B) \\ \cdot P(E \mid C,D) \cdot P(F \mid C,E) \cdot P(H \mid G,F) \end{aligned}$$

$$\boxed{\begin{gathered} P(A,B,C,D,E,F,G,H) = \\ P(A)\,P(B)\,P(G)\,P(C|A,B)\,P(D|B)\,P(E|C,D)\,P(F|C,E)\,P(H|G,F) \end{gathered}}$$

Part (ii)

given each assertion we can use d-separation to determine whether it is True/False.

(a) $B \perp G \mid A$

TRUE

We need to check all paths from B to G. Since G is a root node and it only connects to the rest of the network through H. All paths from B to G must go through H:

- $B \to C \to E \to F \to H \leftarrow G$

- $B \to C \to F \to H \leftarrow G$
- $B \to D \to E \to F \to H \leftarrow G$

For each of these paths we have a v-structure (head-to-head) at H and since both F and G point into H:

$$F \to H \leftarrow G$$

For a v-structure to be active either H or one of its descendants must be observed. Since we only condition on A and H has no descendants this v-structure has to be blocked.

Therefore, because all paths between B and G are blocked given A:

$$\boxed{B \perp G \mid A}$$

(b) $C \perp D \mid F$

FALSE

Given our path:

$$C \leftarrow B \to D$$

Since this is a fork structure at B, and for a fork $X \leftarrow Y \to Z$ the path is blocked if and only if the middle node Y is observed.

Here, the middle node is B. Since we condition only on F (not on B), the node B is not observed and therefore this path remains active.

Since there exists an active path between C and D given F:

$$\boxed{C \not\perp D \mid F}$$

(c) $C \perp D \mid A$

FALSE

If you consider the path:

$$C \leftarrow B \to D$$

This is the same fork structure at B. Since we condition only on A (not on B), the node B is not observed, and this path remains active.

And dince there exists an active path between C and D given A:

$$\boxed{(C \not\perp D \mid A)}$$

(d) $H \perp B \mid C, F$

TRUE

We will check all paths from H to B:

1. Path**:** $H \leftarrow G$

   This path ends at G (root) and never reaches B. Not important.

2. Path**:** $H \leftarrow F \leftarrow C \leftarrow B$

   This is a chain. Since F is observed the path is blocked at F.

3. Path: $H \leftarrow F \leftarrow E \leftarrow C \leftarrow B$

   Since F is observed the path is blocked at F.

4. Path: $H \leftarrow F \leftarrow E \leftarrow D \leftarrow B$

   Since F is **observed**, the path is **blocked** at F.

5. Path: $H \leftarrow F \leftarrow C \leftarrow A$ (does not reach B)

and sicne all paths from H to B must pass through F which is observed. Therefore, all paths are blocked. Since all paths between H and B are blocked given $\{C, F\}$:

$$\boxed{H \perp B \mid C, F}$$

(e) "There is at least one node in this BN such that all other nodes are in its Markov Blanket"

FALSE

Since the Markov Blanket of a node X consists of:

- Parents of X
- Children of X
- Co-parents (other parents of X's children)

We can compute the Markov Blanket for each node using the corrected graph structure:

Node	Parents	Children	Co-parents	Markov Blanket	Missing
A	$\emptyset$	$\{C\}$	$\{B\}$	$\{B, C\}$	D, E, F, G, H
B	$\emptyset$	$\{C, D\}$	$\{A\}$	$\{A, C, D\}$	E, F, G, H
C	$\{A, B\}$	$\{E, F\}$	$\{D, E\}^*$	$\{A, B, D, E, F\}$	G, H
D	$\{B\}$	$\{E\}$	$\{C\}$	$\{B, C, E\}$	A, F, G, H
E	$\{C, D\}$	$\{F\}$	$\{C\}^*$	$\{C, D, F\}$	A, B, G, H
F	$\{C, E\}$	$\{H\}$	$\{G\}$	$\{C, E, G, H\}$	A, B, D
G	$\emptyset$	$\{H\}$	$\{F\}$	$\{F, H\}$	A, B, C, D, E
H	$\{G, F\}$	$\emptyset$	$\emptyset$	$\{F, G\}$	A, B, C, D, E

* **E is both a child of C and a co-parent (for child F) so counted once. D is a co-parent for child E.*

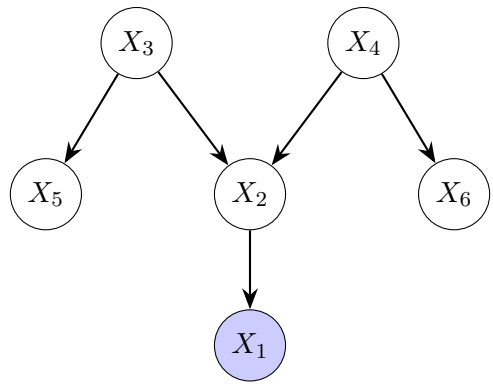
The node with the largest Markov Blanket is C, with:

$$\text{MB}(C) = \{A, B, D, E, F\}$$

This contains 5/7 other nodes but G and H are missing from C's Markov Blanket.
Since no node has all 7 other nodes in its Markov Blanket, the statement is **FALSE**.

Problem 2

Given the Bayesian network where X_1 is observed (shaded):



Questions 1-3

1. $P(X_2 = x_2 | X_1 = x_1)$

Since X_5 and X_6 are not ancestors of X_1, they factor out completely:

$$\boxed{P(X_2 = x_2 | X_1 = x_1) = \frac{\sum_{x_3,x_4} P(X_3)P(X_4)P(X_2 = x_2|X_3, X_4)P(X_1 = x_1|X_2 = x_2)}{\sum_{x_2',x_3,x_4} P(X_3)P(X_4)P(X_2'|X_3, X_4)P(X_1 = x_1|X_2')}}$$

2. $P(X_3 = x_3 | X_1 = x_1)$

$$\boxed{P(X_3 = x_3 | X_1 = x_1) = \frac{\sum_{x_2,x_4} P(X_3 = x_3)P(X_4)P(X_2|X_3 = x_3, X_4)P(X_1 = x_1|X_2)}{\sum_{x_2,x_3',x_4} P(X_3')P(X_4)P(X_2|X_3', X_4)P(X_1 = x_1|X_2)}}$$

3. $P(X_5 = x_5 | X_1 = x_1)$

X_5 is connected to X_1 through the path $X_5 \leftarrow X_3 \rightarrow X_2 \rightarrow X_1$. Observing X_1 gives information about X_2, then X_3, then X_5.

$$\boxed{P(X_5 = x_5 | X_1 = x_1) = \frac{\sum_{x_2,x_3,x_4} P(X_3)P(X_4)P(X_5 = x_5|X_3)P(X_2|X_3, X_4)P(X_1 = x_1|X_2)}{\sum_{x_2,x_3,x_4,x_5'} P(X_3)P(X_4)P(X_5'|X_3)P(X_2|X_3, X_4)P(X_1 = x_1|X_2)}}$$

Questions 4-6

4. $P(X_2 = x_2 | X_1 = x_1, X_3 = x_3, X_4 = x_4, X_5 = x_5, X_6 = x_6)$

$\text{MB}(X_2) = \{X_1, X_3, X_4\}$ (parents: X_3, X_4; child: X_1; co-parents of X_1: none)

$$\boxed{P(X_2 = x_2 | X_1, X_3, X_4, X_5, X_6) = \frac{P(X_2 = x_2|X_3, X_4) \cdot P(X_1|X_2 = x_2)}{\sum_{x_2'} P(X_2'|X_3, X_4) \cdot P(X_1|X_2')}}$$

5. $P(X_3 = x_3 | X_1 = x_1, X_2 = x_2, X_4 = x_4, X_5 = x_5, X_6 = x_6)$

$\mathrm{MB}(X_3) = \{X_2, X_4, X_5\}$ (parents: none; children: X_2, X_5; co-parents of X_2: X_4)

$$\boxed{P(X_3 = x_3 | X_1, X_2, X_4, X_5, X_6) = \frac{P(X_3 = x_3) \cdot P(X_5 | X_3 = x_3) \cdot P(X_2 | X_3 = x_3, X_4)}{\sum_{x_3'} P(X_3') \cdot P(X_5 | X_3') \cdot P(X_2 | X_3', X_4)}}$$

6. $P(X_5 = x_5 | X_1 = x_1, X_2 = x_2, X_3 = x_3, X_4 = x_4, X_6 = x_6)$

$\mathrm{MB}(X_5) = \{X_3\}$ (parent: X_3; children: none; co-parents: none)

Since X_5 has no children and we're conditioning on its only parent X_3, this is just the CPD:

$$\boxed{P(X_5 = x_5 | X_1, X_2, X_3, X_4, X_6) = P(X_5 = x_5 | X_3)}$$

Problem 3

Part (a)

Claim: G_1 and G_2 having the same skeleton is a *necessary*, but not *sufficient* condition for I-equivalence.

Proof of Necessity:
If G_1 and G_2 are I-equivalent, they must encode the same set of conditional independencies. If the skeletons differ, then there exists an edge (X, Y) in one graph but not the other. This means:

- In the graph with the edge: X and Y are *not* unconditionally independent
- In the graph without the edge: X and Y *could be* independent (no direct connection)

This would lead to different independence assertions, so they cannot be I-equivalent. Therefore, same skeleton is **necessary**.

Proof that it's not Sufficient (Counterexample):
Consider three nodes A, B, C with the same skeleton $A - B - C$ (a chain).

G_1: $A \to B \to C$ $\qquad$ G_2: $A \to B \leftarrow C$

Both have skeleton $A - B - C$, but:

- In G_1: $A \perp C \mid B$ (chain — conditioning on middle blocks the path)
- In G_2: $A \not\perp C \mid B$ (v-structure — conditioning on middle *opens* the path)

Since they encode different conditional independencies, G_1 and G_2 are **not** I-equivalent despite having the same skeleton.

Part (b)

Claim: G_1 and G_2 having the same skeleton and same v-structures is a *sufficient*, but not *necessary* condition for I-equivalence.

Proof of Sufficiency:
Two BNs are I-equivalent if and only if they encode the same d-separation statements. D-separation depends on:

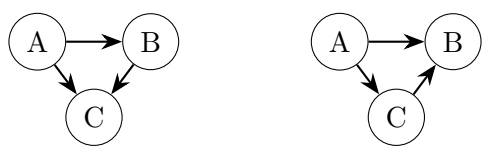
1. The paths between nodes (determined by the **skeleton**)
2. Whether paths are blocked/unblocked, which depends on:
   - Chains and forks: blocked when middle node is observed
   - V-structures: blocked when middle node is *not* observed (and no descendants observed)

If two graphs have the same skeleton and the same v-structures, then:

- They have identical paths between any two nodes
- the remaining triples are all chains or forks, which have the same blocking behavior regardless of edge direction
- all triples are in the same locations and behave identically

Therefore, they produce identical d-separation statements, making them I-equivalent. **Same skeleton + same v-structures is sufficient.**

Proof that it's not Necessary (Counterexample):
Consider a fully connected graph on 3 nodes where every pair has an edge:



G_1 G_2

G_1 has a v-structure at C (both A and B point to C), while G_2 does not (at C, only A points to it; B receives from C).
However, in a fully connected graph, **no two nodes are ever conditionally independent** given any subset — there's always a direct edge. So both graphs encode $I(G_1) = I(G_2) = \emptyset$ (no independence statements).
Therefore G_1 and G_2 are I-equivalent even though they have **different v-structures**. This shows same v-structures is **not necessary**.

Problem 4

Part 1: Derive $\psi_{ij}(x_i, x_j)$ and $\psi_i(x_i)$

The multivariate Gaussian density is:

$$P(\mathbf{X}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{X} - \boldsymbol{\mu})^T \boldsymbol{\Omega}(\mathbf{X} - \boldsymbol{\mu})\right)$$

Expanding the quadratic form in the exponent:

$$(\mathbf{X} - \boldsymbol{\mu})^T \boldsymbol{\Omega}(\mathbf{X} - \boldsymbol{\mu}) = \sum_{i=1}^{d} \sum_{j=1}^{d} \Omega_{ij}(x_i - \mu_i)(x_j - \mu_j)$$

This can be split into diagonal terms ($i = j$) and off-diagonal terms ($i \neq j$):

$$= \sum_{i=1}^{d} \Omega_{ii}(x_i - \mu_i)^2 + \sum_{i \neq j} \Omega_{ij}(x_i - \mu_i)(x_j - \mu_j)$$

For the factorization $P(\mathbf{X}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) \propto \prod_{(i,j)\in E} \psi_{ij}(x_i, x_j) \prod_{i \in V} \psi_i(x_i)$:

Node potentials:

$$\psi_i(x_i) = \exp\left(-\frac{1}{2}\Omega_{ii}(x_i - \mu_i)^2\right)$$

Edge potentials:

$$\psi_{ij}(x_i, x_j) = \exp\left(-\Omega_{ij}(x_i - \mu_i)(x_j - \mu_j)\right)$$

Note: The factor of $\frac{1}{2}$ is absorbed differently for edges dince each unordered pair (i, j) appears once in the product over edges, but the sum has both $\Omega_{ij}(x_i - \mu_i)(x_j - \mu_j)$ and $\Omega_{ji}(x_j - \mu_j)(x_i - \mu_i)$, and $\Omega_{ij} = \Omega_{ji}$ by symmetry.

Part 2: Show $(i, j) \notin E \Leftrightarrow X_i \perp X_j | X_{V\setminus\{i,j\}}$

We rewrite the distribution using edge-only factors:

$$P(\mathbf{X}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) \propto \prod_{(i,j)\in E} \psi'_{ij}(x_i, x_j)$$

where

$$\psi'_{ij}(x_i, x_j) = \psi_{ij}(x_i, x_j) \times \psi_i(x_i)^{1/n(i)} \times \psi_j(x_j)^{1/n(j)}$$

and $n(i)$ is the number of neighbors of node i.

Theorem: For a multivariate Gaussian, $X_i \perp X_j | X_{V\setminus\{i,j\}}$ if and only if $\Omega_{ij} = 0$.

Proof of the equivalence:

($\Rightarrow$) If $(i, j) \notin E$, then:

- There is no edge factor ψ'_{ij} connecting x_i and x_j directly
- By the pairwise Markov property for undirected graphical models, non-adjacent nodes are conditionally independent given all other nodes
- Therefore $X_i \perp X_j | X_{V\setminus\{i,j\}}$

($\Leftarrow$) If $X_i \perp X_j | X_{V \setminus \{i,j\}}$, then:

- For Gaussians, this conditional independence holds iff $\Omega_{ij} = 0$
- When $\Omega_{ij} = 0$, the edge potential $\psi_{ij}(x_i, x_j) = \exp(0) = 1$
- A constant factor of 1 means no interaction between x_i and x_j
- So we can remove edge (i, j) from E without changing the distribution

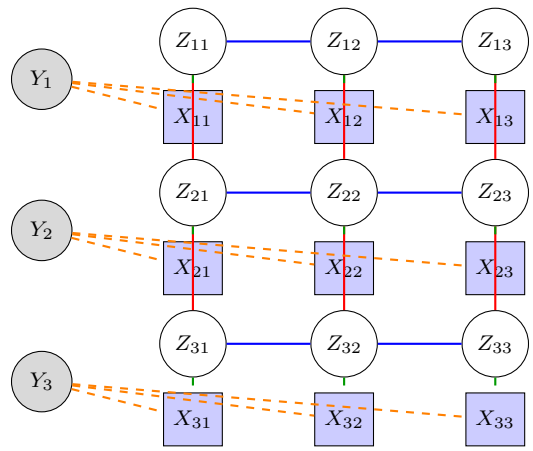
Conclusion: $(i, j) \notin E \Leftrightarrow \Omega_{ij} = 0 \Leftrightarrow X_i \perp X_j | X_{V \setminus \{i,j\}}$
The zeros in the precision matrix $\mathbf{\Omega}$ directly encode the missing edges in the graphical model, and each missing edge corresponds to a conditional independence.

Problem 5

We design an undirected graphical model for denoising printer output with:

- $Z \in \{0,1\}^{m \times n}$: unobserved true output (1 = black, 0 = white)
- $Y \in \{0,1\}^{m}$: unobserved row stuck indicators
- $X \in \{0,1\}^{m \times n}$: observed noisy output

Part 1



Legend: Blue = ψ_β (horizontal), Red = ψ_γ (vertical), Green dashed = ψ_θ (Z-X), Orange dashed = ψ_μ (Y-X). Shaded nodes (X_{ij}) are observed.

Parameter Intuitions:

- $\alpha \approx 1$: No strong prior on black vs white pixels
- $\beta > 1$: Horizontal smoothness — adjacent pixels likely same color
- $\gamma > 1$: Vertical smoothness — adjacent pixels likely same color
- $\theta > 1$: Observation accuracy — output usually matches true value
- $\mu < 1$: When row is stuck ($Y_i = 1$), seeing black ($X_{ij} = 1$) is less likely

Part 2

When conditioning on X, we have a CRF:

Partition Function:

$$\mathcal{Z}(X) = \sum_{Y \in \{0,1\}^m} \sum_{Z \in \{0,1\}^{m \times n}} \left[\prod_{i,j} \alpha^{\mathbf{1}\{Z_{ij}=1\}}\right] \times \left[\prod_{i,j} \beta^{\mathbf{1}\{Z_{ij}=Z_{i,j+1}\}}\right] \times \left[\prod_{i,j} \gamma^{\mathbf{1}\{Z_{ij}=Z_{i+1,j}\}}\right] \times \left[\prod_{i,j} \theta^{\mathbf{1}\{X_{ij}=Z_{ij}\}}\right] \times \left[\prod_{i,j} \mu^{\mathbf{1}\{X_{ij}=1 \wedge Y_i=1\}}\right]$$

Part 3

Given:

$$X = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \quad Y = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Counting the factors:

- $\#\{Z_{ij} = 1\} = 6$
- $\#\{Z_{ij} = Z_{i,j+1}\} = 4$
- $\#\{Z_{ij} = Z_{i+1,j}\} = 3$
- $\#\{X_{ij} = Z_{ij}\} = 7$
- $\#\{X_{ij} = 1 \wedge Y_i = 1\} = 1$

(a) $\alpha = 1.3, \beta = 1.3, \gamma = 1.4, \theta = 1.0, \mu = 0.8$:

$$\tilde{P}_a = 1.3^6 \times 1.3^4 \times 1.4^3 \times 1.0^7 \times 0.8^1 = \mathbf{30.263}$$

(b) $\alpha = 0.5, \beta = 1.5, \gamma = 1.0, \theta = 0.8, \mu = 1.2$:

$$\tilde{P}_b = 0.5^6 \times 1.5^4 \times 1.0^3 \times 0.8^7 \times 1.2^1 = \mathbf{0.0199}$$

Answer: Parameter setting **(a)** leads to a greater conditional data likelihood.
$\tilde{P}_a / \tilde{P}_b \approx 1520$

Code:

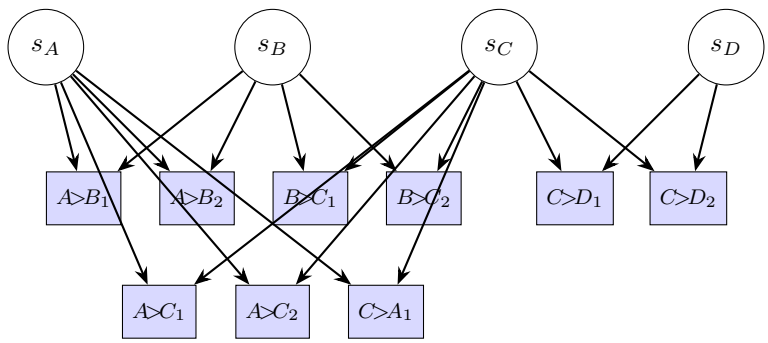
```
import numpy as np

def calc_likelihood(X, Y, Z, a, b, g, th, mu):
    m, n = X.shape

    # count Z=1
    n_black = np.sum(Z == 1)

    # horiz neighbors matching
    n_h = 0
    for i in range(m):
        for j in range(n-1):
            if Z[i,j] == Z[i,j+1]:
                n_h += 1

    # vert neighbors matching
    n_v = 0
    for i in range(m-1):
        for j in range(n):
            if Z[i,j] == Z[i+1,j]:
                n_v += 1

    # X matches Z
    n_obs = np.sum(X == Z)
```

```
    # stuck row & black pixel
    n_stuck = 0
    for i in range(m):
        if Y[i] == 1:
            n_stuck += np.sum(X[i] == 1)

    L = (a**n_black) * (b**n_h) * (g**n_v) * (th**n_obs) * (mu**n_stuck)

    return L, {'black': n_black, 'horiz': n_h, 'vert': n_v, 'obs': n_obs,
       'stuck': n_stuck}


if __name__ == "__main__":
    X = np.array([[0,0,1], [0,1,1], [0,0,1]])
    Y = np.array([1,0,0])
    Z = np.array([[0,1,1], [1,1,1], [0,0,1]])

    # setting a
    La, ca = calc_likelihood(X, Y, Z, a=1.3, b=1.3, g=1.4, th=1.0, mu=0.8)
    print(f"(a): {La:.4f}, counts={ca}")

    # setting b
    Lb, cb = calc_likelihood(X, Y, Z, a=0.5, b=1.5, g=1.0, th=0.8, mu=1.2)
    print(f"(b): {Lb:.4f}, counts={cb}")

    print(f"\nratio a/b = {La/Lb:.2f}")
    print(f"winner: {'(a)' if La > Lb else '(b)'}")
```

Problem 6

6.1

The BN has four skill-level nodes s_A, s_B, s_C, s_D (each taking values $1, \ldots, 10$) as root nodes, and game outcome nodes that depend on the skill levels of the two participating players. Since skill levels are a priori independent, and all games are independent given skill levels, each game outcome node has exactly two parents: the skill levels of the two players involved.



Each game outcome node G is a binary variable (win/lose). Given skill levels s_X, s_Y, the CPD is:

$$P(X \text{ beats } Y \mid s_X, s_Y) = \frac{1}{1 + \exp(s_Y - s_X)}$$

The prior on each skill level is uniform: $P(s_i = k) = \frac{1}{10}$ for $k \in \{1, \ldots, 10\}$.

The joint factorizes as: $P(\text{all}) = \prod_{i \in \{A,B,C,D\}} P(s_i) \cdot \prod_{\text{games}} P(\text{outcome} \mid s_{\text{player1}}, s_{\text{player2}})$

6.2

NO the skill levels are **not** a posteriori independent.

In the belief network, each game outcome node is a v-structure (collider) with two skill-level parents. For example, in the game "A beats B", the node has parents s_A and s_B:

$$s_A \rightarrow (\text{A beats B}) \leftarrow s_B$$

By d-separation rules, observing a v-structure node (or its descendants) *activates* the path between the parents. Since all game outcomes are observed, the paths between all skill-level nodes that share a common game become active. This creates dependencies:

- s_A and s_B become dependent (through A-vs-B games)
- s_B and s_C become dependent (through B-vs-C games)
- s_A and s_C become dependent (through A-vs-C games)
- s_C and s_D become dependent (through C-vs-D games)
- s_A and s_D become dependent (transitively through s_C)

We can verify this numerically. Let the joint posterior over (s_A, s_B) be $P(s_A, s_B \mid \text{data})$. If independent, this should equal $P(s_A \mid \text{data}) \cdot P(s_B \mid \text{data})$. Computing both:

$$\max_{s_A, s_B} |P(s_A, s_B \mid \text{data}) - P(s_A \mid \text{data}) P(s_B \mid \text{data})| = 0.0412 \neq 0$$

confirming the skill levels are **not** a posteriori independent.

6.3

We compute:

$$P(D \text{ beats } A \mid \text{data}) = \sum_{s_A, s_B, s_C, s_D} P(D \text{ beats } A \mid s_D, s_A) \cdot P(s_A, s_B, s_C, s_D \mid \text{data})$$

where:

$$P(s_A, s_B, s_C, s_D \mid \text{data}) \propto \prod_i P(s_i) \cdot (\text{likelihood of all game outcomes})$$

We enumerate all $10^4 = 10{,}000$ combinations of skill levels and compute the unnormalized posterior then normalize then compute the expectation.

$$P(D \text{ beats } A \mid \text{data}) = \mathbf{0.0450}$$

6.4

The posterior expected skill level of each player is:

$$E[s_i \mid \text{data}] = \sum_{k=1}^{10} k \cdot P(s_i = k \mid \text{data})$$

Computing the marginal posterior for each player and taking the expectation:

$$E[s_A \mid \text{data}] = \mathbf{7.8671}$$
$$E[s_B \mid \text{data}] = \mathbf{6.8493}$$
$$E[s_C \mid \text{data}] = \mathbf{5.9844}$$
$$E[s_D \mid \text{data}] = \mathbf{3.0461}$$

Code:

```
import numpy as np
def p_win(sA, sB):
    return 1.0 / (1.0 + np.exp(sB - sA))

skills = list(range(1, 11))
posterior = np.zeros((10, 10, 10, 10))

for iA, sA in enumerate(skills):
    for iB, sB in enumerate(skills):
        for iC, sC in enumerate(skills):
            for iD, sD in enumerate(skills):
                likelihood = (
                    p_win(sA, sB) ** 2 *     # A beat B twice
                    p_win(sB, sC) ** 2 *     # B beat C twice
                    p_win(sA, sC) ** 2 *     # A beat C twice
                    p_win(sC, sA) ** 1 *     # C beat A once
                    p_win(sC, sD) ** 2       # C beat D twice (D lost to C
                        twice)
                )
```

```
                posterior[iA, iB, iC, iD] = likelihood  # prior is uniform

Z = posterior.sum()
posterior /= Z

#6.2: Are skill levels a posteriori independent?
#see if P(sA, sB | data) = P(sA | data) * P(sB | data)
marginal_A = posterior.sum(axis=(1, 2, 3))
marginal_B = posterior.sum(axis=(0, 2, 3))
marginal_C = posterior.sum(axis=(0, 1, 3))
marginal_D = posterior.sum(axis=(0, 1, 2))

#see joint of A,B vs product of marginals
joint_AB = posterior.sum(axis=(2, 3))  # P(sA, sB | data)
product_AB = np.outer(marginal_A, marginal_B)
max_diff_AB = np.max(np.abs(joint_AB - product_AB))
print(f"Max␣|P(sA,sB|data)␣-␣P(sA|data)P(sB|data)|␣=␣{max_diff_AB:.6f}")
print(f"Skill␣levels␣are␣NOT␣a␣posteriori␣independent.\n")

#6.3: P(D beats A | data)
p_D_beats_A = 0.0
for iA, sA in enumerate(skills):
    for iD, sD in enumerate(skills):
        p_D_beats_A += p_win(sD, sA) * posterior.sum(axis=(1, 2))[iA, iD]

#P(D beats A | data) = sum_{sA,sB,sC,sD} P(D beats A | sA, sD) * P(sA,sB,
   sC,sD | data)
p_D_beats_A = 0.0
for iA, sA in enumerate(skills):
    for iB, sB in enumerate(skills):
        for iC, sC in enumerate(skills):
            for iD, sD in enumerate(skills):
                p_D_beats_A += p_win(sD, sA) * posterior[iA, iB, iC, iD]

print(f"6.3:␣P(D␣beats␣A␣|␣data)␣=␣{p_D_beats_A:.6f}\n")

#6.4: Posterior expected skill levels
E_sA = sum(sA * marginal_A[iA] for iA, sA in enumerate(skills))
E_sB = sum(sB * marginal_B[iB] for iB, sB in enumerate(skills))
E_sC = sum(sC * marginal_C[iC] for iC, sC in enumerate(skills))
E_sD = sum(sD * marginal_D[iD] for iD, sD in enumerate(skills))

print("6.4:␣Posterior␣expected␣skill␣levels:")
print(f"␣␣E[sA␣|␣data]␣=␣{E_sA:.4f}")
print(f"␣␣E[sB␣|␣data]␣=␣{E_sB:.4f}")
print(f"␣␣E[sC␣|␣data]␣=␣{E_sC:.4f}")
print(f"␣␣E[sD␣|␣data]␣=␣{E_sD:.4f}")

print("\nMarginal␣distributions:")
for name, marg in [("A", marginal_A), ("B", marginal_B), ("C", marginal_C)
   , ("D", marginal_D)]:
    print(f"␣␣P(s{name}=k␣|␣data):")
    for k in range(10):
        print(f"␣␣␣␣k={k+1}:␣{marg[k]:.6f}")
```

Problem 7

We use a Markov Network with energy function:

$$E(\mathbf{Z} = \mathbf{z}, \mathbf{X} = \mathbf{x}) = h \sum_i z_i - \beta \sum_{\{i,j\}} z_i z_j - \nu \sum_i z_i x_i$$

where $z_i, x_i \in \{+1, -1\}$, $h \in \mathbb{R}$, $\beta \in \mathbb{R}_+$, $\nu \in \mathbb{R}_+$.

We apply ICM (Iterated Conditional Modes): initialize $Z = X$ (noisy image), then for each pixel z_i, pick the value in $\{+1, -1\}$ that minimizes the local energy contribution:

$$E_{\text{local}}(z_i) = h \cdot z_i - \beta \cdot z_i \sum_{j \in \mathcal{N}(i)} z_j - \nu \cdot z_i \cdot x_i$$

The optimal update is $z_i = \text{sign}\Big(\beta \sum_{j \in \mathcal{N}(i)} z_j + \nu \cdot x_i - h\Big)$.

Results for Three Parameter Settings

The noisy image has an error rate of **9.94%** (fraction of flipped pixels).

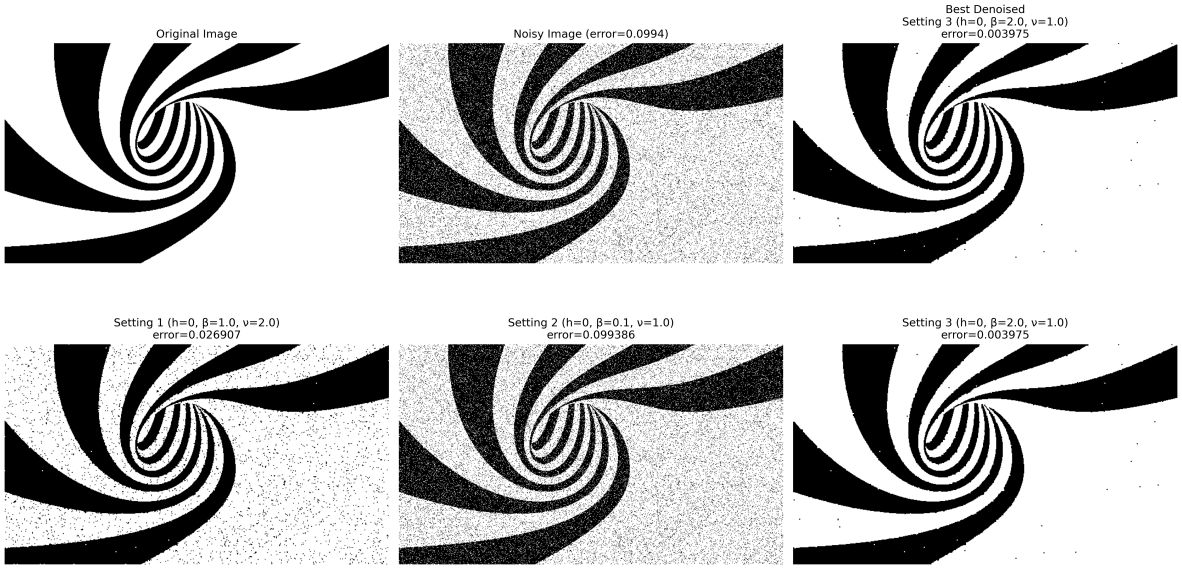
Setting	h	β	ν	**Error Rate**	**Iterations**
1	0.0	1.0	2.0	2.69%	5
2	0.0	0.1	1.0	9.94% (no improvement)	1
3 (best)	**0.0**	**2.0**	**1.0**	**0.40%**	6

Analysis:

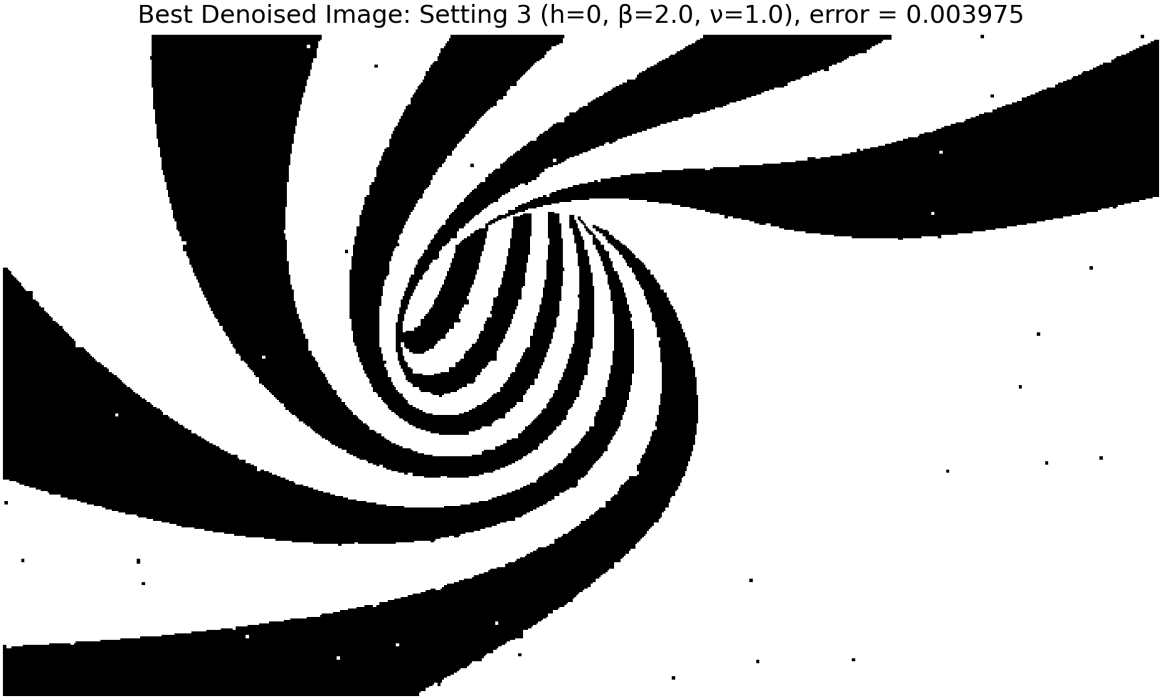
- **Setting 1** ($\beta = 1.0, \nu = 2.0$): Moderate smoothing with strong observation weight. Reduces error from 9.94% to 2.69%.
- **Setting 2** ($\beta = 0.1, \nu = 1.0$): Very weak smoothing. The β term is too small relative to ν, so each pixel essentially stays at its noisy value. No pixels are flipped and the error rate remains 9.94%.
- **Setting 3** ($\beta = 2.0, \nu = 1.0$): Strong smoothing with moderate observation weight. The higher β encourages neighboring pixels to agree, which is very effective for this image (which has large uniform regions). Error drops to just 0.40%.

The ratio β/ν controls the tradeoff between smoothness and fidelity to the noisy observations. A higher ratio leads to more smoothing which helps since real images have spatial coherence.

Best Denoised Image



Best Denoised Image: Setting 3 (h=0, β=2.0, ν=1.0), error = 0.003975



Code:

```
import numpy as np
import scipy.io
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

data = scipy.io.loadmat(r"/Users/skarjagi6/Library/CloudStorage/Dropbox-
    GaTech/Shreesh␣Karjagi/coursework/GRAPH_ML/HWK1/Data/hw1-Prob7_images.
    mat")
```

```
origIm = data['origImg'].astype(np.float64)   # shape (400, 700), values
    in {-1, +1}
noisyImg = data['noisyImg'].astype(np.float64) # shape (400, 700), values
    in {-1, +1}

rows, cols = origIm.shape
print(f"Image␣size:␣{rows}␣x␣{cols}")
print(f"Noise␣rate:␣{np.mean(origIm␣!=␣noisyImg):.4f}")


def compute_energy(Z, X, h, beta, nu):
    E = h * np.sum(Z)
    E -= beta * np.sum(Z[:, :-1] * Z[:, 1:])
    E -= beta * np.sum(Z[:-1, :] * Z[1:, :])
    E -= nu * np.sum(Z * X)

    return E


def denoise_icm(X, h, beta, nu, max_iters=20):
    Z = X.copy()  # int Z to noisy values

    for iteration in range(max_iters):
        num_flipped = 0
        energy = compute_energy(Z, X, h, beta, nu)

        for i in range(rows):
            for j in range(cols):
                # sum of neighbors
                S = 0.0
                if i > 0:       S += Z[i-1, j]
                if i < rows-1:  S += Z[i+1, j]
                if j > 0:       S += Z[i, j-1]
                if j < cols-1:  S += Z[i, j+1]

                val = beta * S + nu * X[i, j] - h
                new_z = 1.0 if val > 0 else -1.0

                if new_z != Z[i, j]:
                    num_flipped += 1
                    Z[i, j] = new_z

        new_energy = compute_energy(Z, X, h, beta, nu)
        print(f"␣␣Iter␣{iteration+1}:␣flipped␣{num_flipped}␣pixels,␣energy
           ␣=␣{new_energy:.2f}")

        if num_flipped == 0:
            print(f"␣␣Converged␣at␣iteration␣{iteration+1}")
            break

    return Z


def error_rate(Z, orig):
```

```
    return np.mean(Z != orig)


settings = [
    {"h": 0.0, "beta": 1.0, "nu": 2.0,  "label": "Setting␣1␣(h=0,␣  =1.0,␣
         =2.0)"},
    {"h": 0.0, "beta": 0.1, "nu": 1.0,  "label": "Setting␣2␣(h=0,␣  =0.1,␣
         =1.0)"},
    {"h": 0.0, "beta": 2.0, "nu": 1.0,  "label": "Setting␣3␣(h=0,␣  =2.0,␣
         =1.0)"},
]

results = []

for idx, s in enumerate(settings):
    print(f"\n{'='*60}")
    print(f"{s['label']}")
    print(f"{'='*60}")

    Z = denoise_icm(noisyImg, s['h'], s['beta'], s['nu'], max_iters=15)
    err = error_rate(Z, origImg)
    results.append((s['label'], err, Z.copy()))
    print(f"Error␣rate:␣{err:.6f}␣({err*100:.2f}%)")
best_idx = np.argmin([r[1] for r in results])
print(f"\n{'='*60}")
print(f"BEST:␣{results[best_idx][0]}␣with␣error␣rate␣{results[best_idx
   ][1]:.6f}")
print(f"{'='*60}")

noisy_err = error_rate(noisyImg, origImg)
print(f"Noisy␣image␣error␣rate␣(baseline):␣{noisy_err:.6f}␣({noisy_err
   *100:.2f}%)")

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

axes[0, 0].imshow(origImg, cmap='gray', vmin=-1, vmax=1)
axes[0, 0].set_title('Original␣Image')
axes[0, 0].axis('off')

axes[0, 1].imshow(noisyImg, cmap='gray', vmin=-1, vmax=1)
axes[0, 1].set_title(f'Noisy␣Image␣(error={noisy_err:.4f})')
axes[0, 1].axis('off')

axes[0, 2].imshow(results[best_idx][2], cmap='gray', vmin=-1, vmax=1)
axes[0, 2].set_title(f'Best␣Denoised\n{results[best_idx][0]}\nerror={
   results[best_idx][1]:.6f}')
axes[0, 2].axis('off')

for i, (label, err, Z) in enumerate(results):
    axes[1, i].imshow(Z, cmap='gray', vmin=-1, vmax=1)
    axes[1, i].set_title(f'{label}\nerror={err:.6f}')
    axes[1, i].axis('off')

plt.tight_layout()
```

```
plt.savefig('/Users/skarjagi6/Library/CloudStorage/Dropbox-GaTech/Shreesh␣
    Karjagi/coursework/GRAPH_ML/HWK1/prob7_denoising.png', dpi=300,
    bbox_inches='tight')
plt.close()

fig2, ax2 = plt.subplots(1, 1, figsize=(10, 6))
ax2.imshow(results[best_idx][2], cmap='gray', vmin=-1, vmax=1)
ax2.set_title(f'Best␣Denoised␣Image:␣{results[best_idx][0]},␣error␣=␣{
    results[best_idx][1]:.6f}')
ax2.axis('off')
plt.savefig('/Users/skarjagi6/Library/CloudStorage/Dropbox-GaTech/Shreesh␣
    Karjagi/coursework/GRAPH_ML/HWK1/prob7_best.png', dpi=300, bbox_inches=
    'tight')
plt.close()
```

Problem 8

8.1

Step 1

Words with $\text{Count}(x) < 5$ in the training data are replaced with `_RARE_`. The modified training file is then used to collect counts.

Step 2

The emission parameters are computed as:

$$e(x \mid y) = \frac{\text{Count}(y \to x)}{\text{Count}(y)}$$

Step 3

For each word x in the test set, we assign:

$$y^* = \arg\max_y e(x \mid y)$$

For words not seen in training, we use the `_RARE_` emission probabilities.

Baseline Results:

```
precision recall F1-Score GENE: 0.158861 0.660436 0.256116
```

The baseline has high recall (0.66) but very low precision (0.16) it over-predicts GENE tags because many words have higher emission probability for I-GENE than for O, even when they're not actually gene names.

8.2

Transition probabilities

The trigram transition parameters are:

$$q(y_i \mid y_{i-2}, y_{i-1}) = \frac{\text{Count}(y_{i-2}, y_{i-1}, y_i)}{\text{Count}(y_{i-2}, y_{i-1})}$$

Viterbi algorithm

We implement the Viterbi algorithm to find:

$$\arg\max_{y_1,\ldots,y_n} p(x_1, \ldots, x_n, y_1, \ldots, y_n)$$

using the recurrence:

$$\pi(k, u, v) = \max_w [\pi(k-1, w, u) \cdot q(v \mid w, u) \cdot e(x_k \mid v)]$$

with backpointers to recover the optimal tag sequence.

Viterbi Results:

`precision recall F1-Score GENE: 0.541555 0.314642 0.398030`

Adding trigram transitions bumps precision way up (0.16 to 0.54) at the cost of recall, giving a better F1 overall. The recall drops (0.66 to 0.31) because the model is now more conservative about predicting I-GENE tags, but the overall F1 improves substantially.

Code:

```
import sys
import os
import math
from collections import defaultdict

DATA_DIR = r"/Users/skarjagi6/Library/CloudStorage/Dropbox-GaTech/Shreesh␣
    Karjagi/coursework/GRAPH_ML/HWK1/Data/Prob-8-hmm-data"
TRAIN_FILE = os.path.join(DATA_DIR, "gene.train")
TEST_FILE = os.path.join(DATA_DIR, "gene.test")
KEY_FILE = os.path.join(DATA_DIR, "gene.key")
RARE_TRAIN_FILE = os.path.join(DATA_DIR, "gene_rare.train")
COUNTS_FILE = os.path.join(DATA_DIR, "gene.counts")
P1_OUT = os.path.join(DATA_DIR, "gene_test.p1.out")
P2_OUT = os.path.join(DATA_DIR, "gene_test.p2.out")

RARE_THRESHOLD = 5

def get_word_counts(train_file):
    word_counts = defaultdict(int)
    with open(train_file, 'r') as f:
        for line in f:
            line = line.strip()
            if line:
                parts = line.split("␣")
                word = "␣".join(parts[:-1])
                word_counts[word] += 1
    return word_counts


def replace_rare_words(train_file, output_file, threshold=5):
    word_counts = get_word_counts(train_file)

    with open(train_file, 'r') as fin, open(output_file, 'w') as fout:
        for line in fin:
            line_stripped = line.strip()
            if line_stripped:
                parts = line_stripped.split("␣")
                word = "␣".join(parts[:-1])
                tag = parts[-1]
                if word_counts[word] < threshold:
                    fout.write(f"_RARE_␣{tag}\n")
                else:
                    fout.write(line)
            else:
                fout.write("\n")
```

```
def collect_counts(train_file):
    emission_counts = defaultdict(int)  #(word, tag) -> count
    unigram_counts = defaultdict(int)   #(tag,) -> count
    bigram_counts = defaultdict(int)    #(tag1, tag2) -> count
    trigram_counts = defaultdict(int)   #(tag1, tag2, tag3) -> count
    all_tags = set()
    sentences = []
    current = []
    with open(train_file, 'r') as f:
        for line in f:
            line = line.strip()
            if line:
                parts = line.split("␣")
                word = "␣".join(parts[:-1])
                tag = parts[-1]
                current.append((word, tag))
                all_tags.add(tag)
            else:
                if current:
                    sentences.append(current)
                    current = []
    if current:
        sentences.append(current)

    for sent in sentences:
        tags = [tag for word, tag in sent]

        for word, tag in sent:
            emission_counts[(word, tag)] += 1

        #N-gram counts with boundary symbols
        padded = ["*", "*"] + tags + ["STOP"]

        for i in range(len(tags)):
            unigram_counts[(tags[i],)] += 1
        for i in range(len(padded) - 1):
            bigram_counts[(padded[i], padded[i+1])] += 1
        for i in range(len(padded) - 2):
            trigram_counts[(padded[i], padded[i+1], padded[i+2])] += 1
    return emission_counts, unigram_counts, bigram_counts, trigram_counts,
        all_tags


def compute_emission_probs(emission_counts, unigram_counts):
    #e(x|y) = Count(y -> x) / Count(y)
    emission_probs = {}
    for (word, tag), count in emission_counts.items():
        emission_probs[(word, tag)] = count / unigram_counts[(tag,)]
    return emission_probs

def compute_transition_probs(bigram_counts, trigram_counts):
    #q(yi | yi-2, yi-1) = Count(yi-2, yi-1, yi) / Count(yi-2, yi-1)
    transition_probs = {}
```

```
    for (t1, t2, t3), count in trigram_counts.items():
        transition_probs[(t1, t2, t3)] = count / bigram_counts[(t1, t2)]
    return transition_probs

def baseline_tagger(test_file, output_file, emission_probs, all_tags,
   word_set):
    with open(test_file, 'r') as fin, open(output_file, 'w') as fout:
        for line in fin:
            word = line.strip()
            if word:
                lookup = word if word in word_set else "_RARE_"

                best_tag = None
                best_prob = -1.0
                for tag in all_tags:
                    prob = emission_probs.get((lookup, tag), 0.0)
                    if prob > best_prob:
                        best_prob = prob
                        best_tag = tag

                fout.write(f"{word}␣{best_tag}\n")
            else:
                fout.write("\n")

def viterbi(sentence, emission_probs, transition_probs, all_tags, word_set
   ):
    n = len(sentence)
    tags = list(all_tags)

    #pi[k][(u, v)] for k = 1..n
    pi = [defaultdict(float) for _ in range(n + 1)]
    bp = [defaultdict(str) for _ in range(n + 1)]

    #base case: pi[0][("*", "*")] = 1.0
    pi[0][("*", "*")] = 1.0

    def get_emission(word, tag):
        lookup = word if word in word_set else "_RARE_"
        return emission_probs.get((lookup, tag), 0.0)

    def get_transition(t1, t2, t3):
        return transition_probs.get((t1, t2, t3), 0.0)

    def possible_tags(k):
        if k <= 0:
            return ["*"]
        return tags

    for k in range(1, n + 1):
        word = sentence[k - 1]
        e = {}
        for v in possible_tags(k):
            e[v] = get_emission(word, v)
```

```
        for v in possible_tags(k):
            if e[v] == 0:
                continue
            for u in possible_tags(k - 1):
                for w in possible_tags(k - 2):
                    if pi[k - 1].get((w, u), 0.0) == 0:
                        continue
                    q = get_transition(w, u, v)
                    if q == 0:
                        continue
                    score = pi[k - 1][(w, u)] * q * e[v]
                    if score > pi[k].get((u, v), 0.0):
                        pi[k][(u, v)] = score
                        bp[k][(u, v)] = w

    best_score = 0.0
    best_u, best_v = None, None
    for u in possible_tags(n - 1):
        for v in possible_tags(n):
            if pi[n].get((u, v), 0.0) == 0:
                continue
            q_stop = get_transition(u, v, "STOP")
            score = pi[n][(u, v)] * q_stop
            if score > best_score:
                best_score = score
                best_u, best_v = u, v

    if best_u is None:
        return ["O"] * n

    tag_seq = [""] * n
    tag_seq[n - 1] = best_v
    if n >= 2:
        tag_seq[n - 2] = best_u

    for k in range(n - 2, 0, -1):
        tag_seq[k - 1] = bp[k + 2][(tag_seq[k], tag_seq[k + 1])]

    return tag_seq


def viterbi_tagger(test_file, output_file, emission_probs,
    transition_probs, all_tags, word_set):
    sentences = []
    current = []

    with open(test_file, 'r') as f:
        for line in f:
            word = line.strip()
            if word:
                current.append(word)
            else:
                if current:
                    sentences.append(current)
```

```
                    current = []
    if current:
        sentences.append(current)

    with open(output_file, 'w') as fout:
        for i, sent in enumerate(sentences):
            tags = viterbi(sent, emission_probs, transition_probs,
               all_tags, word_set)
            for word, tag in zip(sent, tags):
                fout.write(f"{word}␣{tag}\n")
            fout.write("\n")

            if (i + 1) % 100 == 0:
                print(f"processed␣{i+1}/{len(sentences)}␣sentences")


def evaluate(key_file, pred_file):
    def read_entities(filepath):
        entities = []
        current_start = None
        idx = 0

        with open(filepath, 'r') as f:
            for line in f:
                line = line.strip()
                if line:
                    parts = line.split("␣")
                    tag = parts[-1]

                    if tag.startswith("I-GENE"):
                        if current_start is None:
                            current_start = idx
                    else:
                        if current_start is not None:
                            entities.append((current_start, idx - 1))
                            current_start = None
                    idx += 1
                else:
                    if current_start is not None:
                        entities.append((current_start, idx - 1))
                        current_start = None

        if current_start is not None:
            entities.append((current_start, idx - 1))

        return set(entities)

    gold = read_entities(key_file)
    pred = read_entities(pred_file)

    correct = len(gold & pred)

    precision = correct / len(pred) if pred else 0
    recall = correct / len(gold) if gold else 0
```

```
    f1 = 2 * precision * recall / (precision + recall) if (precision +
       recall) > 0 else 0

    print(f"␣␣Found␣{len(pred)}␣GENEs.␣Expected␣{len(gold)}␣GENEs;␣Correct
       :␣{correct}.")
    print(f"␣␣Precision:␣{precision:.6f}")
    print(f"␣␣Recall:␣␣␣␣{recall:.6f}")
    print(f"␣␣F1-Score:␣␣{f1:.6f}")

    return precision, recall, f1


if __name__ == "__main__":
    os.makedirs(DATA_DIR, exist_ok=True)

    replace_rare_words(TRAIN_FILE, RARE_TRAIN_FILE, RARE_THRESHOLD)

    emission_counts, unigram_counts, bigram_counts, trigram_counts,
       all_tags = \
        collect_counts(RARE_TRAIN_FILE)

    print(f"tags␣found:␣{sorted(all_tags)}")
    print(f"unigram␣counts:␣{dict(unigram_counts)}")
    print(f"emission␣types:␣{len(emission_counts)}")
    print(f"bigrams:␣{len(bigram_counts)}")
    print(f"trigrams:␣{len(trigram_counts)}")

    word_set = set()
    for (word, tag) in emission_counts:
        word_set.add(word)

    emission_probs = compute_emission_probs(emission_counts,
       unigram_counts)

    baseline_tagger(TEST_FILE, P1_OUT, emission_probs, all_tags, word_set)

    print("\nBaseline␣Results:")
    evaluate(KEY_FILE, P1_OUT)

    transition_probs = compute_transition_probs(bigram_counts,
       trigram_counts)
    viterbi_tagger(TEST_FILE, P2_OUT, emission_probs, transition_probs,
       all_tags, word_set)
    evaluate(KEY_FILE, P2_OUT)
```